Interactive FPGA Graphical Calculator

GPT-6 Project Proposal — Group ID 06

Group theme. This project will implement an interactive graphical calculator as one integrated Verilog design on a Basys3 FPGA with a 96×64 OLED display. A unified main menu will let users enter signed values with the board switches and select either a conventional calculator or Graph Mode. The calculator will perform addition, subtraction, multiplication and division, while detecting invalid operations and overflow and retaining recent results. Graph Mode will accept one or two functions through a guided coefficient-entry sequence, render coloured linear or quadratic graphs on scaled axes, and calculate the intersection of two linear functions. Clear status messages will distinguish parallel, identical, invalid and off-screen cases. Modular controller, arithmetic, input and rendering interfaces will support independent testing before final integration. Extension candidates include automatic view fitting, zoom, pan, a movable coordinate cursor, graph tracing, saved function presets, cubic, absolute-value, reciprocal, square-root, step, piecewise and LUT-based sine/cosine graph rendering, a coordinate-value table, automatic root, intercept and turning-point markers, numerical slope readout, memory recall and selectable grid density.

Milla Glännfjord

Group Member 1

Member responsibility. The main responsibility of this group member is to develop a module that reads switches and convert their states into binary numbers, including support for signed values. This group member will also write one module for detecting invalid user inputs and one potential overflow, and implementing a history-data module that stores and supplies the three most recent results to Hanna's calculator interface. The modules will be designed so that the other group members can use it for both the calculator and graphing functions.

Week	Planned work
7	Complete the practice task provided in the project manual to gain a better understanding of the OLED, Verilog programming and the different components needed for the project.
8	Develop the module responsible for reading the switches and interpreting their states as binary numbers. Ensure that the module can handle both positive and negative values and that it can be integrated with the other group members' modules.
9	Implement additional functionality to detect invalid user inputs and potential overflow during calculations. Develop a history module that stores the three most recent calculation results.
10	Have a group meeting to review the progress of the project and to decide how to divide the remaining graphical components between the group members. Discuss how the different modules will be connected and integrated into the complete system.
11	Develop the signed-input preview and history/error panel for the OLED display. Test these graphical components and adjust their interfaces so that they work correctly with the calculator module.
12	Have a final group meeting to integrate all the modules and complete the project. Perform final testing, identify and fix any remaining issues, and ensure that the system meets the project requirements.

Hanna Wahlgren

Group Member 2

Member responsibility. The main responsibility is to implement the calculator module, supporting the four basic arithmetic operations: addition, subtraction, multiplication and division. The calculator will use the FPGA board's buttons to select operations and switches to enter signed binary numbers. Another responsibility is to develop the calculator interface on the OLED display, including digits, mathematical symbols, input values and calculation results. It will also develop the main menu logic in the top module, allowing users to navigate between the calculator and graphing functions, select a program and return to the main menu.

Week	Planned work
7	Completed the practice tasks in the project manual to gain a better understanding of the OLED display and overall Verilog programming required for the project.
8	Started developing the calculator module and implemented the basic arithmetic operations using a state-based approach. Created a function for drawing digits (0–9) on the OLED display and began implementing the graphical representation of input values, mathematical operators and calculation results.
9	Developed the main-menu logic in the top module, allowing users to navigate between the calculator and graphing programs using the buttons. Implemented functionality to select a program and return to the main menu. Worked on integrating Milla's input, validation, overflow and history-data modules into the calculator interface.
10	Test the calculator module and its integration with the other modules. Improve the OLED graphics, including negative numbers, results and error messages. Participate in the group meeting to divide the remaining graphical tasks.
11	Implement the assigned graphical components, including the main-menu interface. Improve the calculator's graphical layout and test navigation between the menu and the different programs.
12	Integrate the calculator and menu with the other group members' modules. Perform final testing on the FPGA board, fix remaining issues and prepare the system for the final demonstration.

Wang Qichen

Group Member 3

Member responsibility. Design and implement the graph-rendering component of our FPGA-based graphical calculator. The renderer will convert the OLED `pixel_index` into coordinates for the 96×64 display and draw a readable graph area with axes, grid lines and labels. It will display two signed linear functions, $y = k_1x + m_1$ and $y = k_2x + m_2$, using different colours and continuous line-drawing logic. Using graph-space results supplied by the controller, it will draw intersection and feature markers and display status messages for parallel, identical, invalid or off-screen cases. During coefficient entry, the OLED will show the input stage and instructions for the board controls.

Week	Planned work
7	Completed the OLED familiarisation tasks and studied RGB565 colours, `pixel_index` conversion and basic Verilog drawing techniques.
8	Investigated coordinate scaling, signed graph values, line-rendering methods and the interface required between the graph controller and OLED renderer.
9	Defined the renderer architecture, colour scheme, graph states, intersection display and exceptional-case overlays, and coordinated the required signals with the controller component.
10	Implement and simulate the core axes, grid, two-line renderer, input-stage text and intersection marker. Add an equation legend and graph-visibility controls, then integrate the OLED renderer.
11	Render zoom, pan and cursor states supplied by the controller, with graph clipping, automatic scale selection, coordinate labels and cursor snapping. Implement a coordinate-value table and selectable grid density, and display controller-supplied root, intercept, turning-point and slope results. Prototype prioritised rendering for quadratic, cubic, absolute-value, reciprocal, square-root, step, piecewise and LUT-based sine/cosine functions. Add adaptive tick labels, asymptote and discontinuity markers, a split graph/table view and parameter animation.
12	Optimise resource usage, verify timing, test signed and extreme inputs on the Basys3, complete group integration and prepare a reliable demonstration. Add saved display presets, a guided demo screen and clearer on-screen help for all graph controls.

Oskar Plogner

Group Member 4

Member responsibility. The main responsibility is to develop the graph controller and mathematical calculations for the graphing mode. This includes managing user inputs through a finite state machine (FSM) for selecting one or two graphs, function types and signed coefficients. Another major responsibility is calculating intersections between linear functions using fixed-point arithmetic, including handling parallel or identical lines, division by zero and overflow. The controller will communicate with the graph renderer by providing coefficients, calculation results and control signals. The controller will also own zoom, pan and cursor state.

Week	Planned work
7	Completed the OLED familiarisation tasks and discussed possible project functionalities and responsibilities within the group.
8	Planned the graphing functionality, including coefficient input, graph selection, intersection calculations and communication between the controller and renderer.
9	Developed the initial `graph_controller.v` module using an FSM. Implemented selection of one or two graphs, linear/quadratic function selection, coefficient storage, reset functionality and signals indicating when the graphs are ready to be plotted. The module has not yet been tested or integrated.
10	Simulate and test the graph controller with different input combinations. Begin integration with the signed-number input module and OLED renderer, adapting the interfaces between the modules. Implement intersection calculations for two linear functions, including handling parallel and identical lines.
11	Implement error handling for invalid calculations, overflow and intersections outside the visible graph area. Continue testing and improving integration with the other modules. Implement and output zoom, pan and cursor state for Wang's renderer. Add function selection, parameters and fixed-point evaluation for prioritised non-linear graphs, together with quadratic/cubic intersection solving, tangent/normal calculations, area estimation, discontinuity detection and parameter-animation control.
12	Integrate the graph controller and intersection calculations with the other modules. Test the complete graphing functionality on the Basys3 board, fix remaining issues and help prepare the final demonstration.